MIE1517 Sample Midterm Questions

Part 1: Introduction

"Training": (whether you "know" the material)

1. What is the difference between artificial intelligence, machine learning, and deep learning?
2. What is the difference between supervised and unsupervised learning?
3. What is the difference between regression and classification?
4. A neuron has a cell body, axons, and dendrites. What order does information flow through these parts?
5. What is a synapse?
6. What does "feed-forward" mean in the context of an artificial neural network?
7. What does "fully-connect" mean in the context of an artificial neural network?
8. A loss function takes two arguments. What are they?
9. Which of the following are ~~*not*~~ steps in neural network training?
    - computing the prediction
    - computing the loss function
    - resetting the parameters to random
    - displaying the image
10. Why do we need both a training set and a test set?
11. Gradient descent is an example of . . .
    - a loss function
    - an optimizer
    - a neural network architecture
    - a type of neuron unit

Generalization: (whether you can apply the material)

1. A researcher wishes to train a neural network to play the game Mario. Is this problem an example of supervised, unsupervised, or reinforcement learning?
2. A researcher wishes to train a neural network to play the game Mario. The researcher rewarded the network based on the amount of time (in seconds) that Mario stayed alive. Would you expect the network to perform well? Why or why not?
3. How is an artificial neural network similar to a biological neural network? How are they different?
4. Consider the forward function of a neural network defined using PyTorch:

```
def forward(self, img):
    return self.layer5(img.view(-1, 256 * 256))
```

- What is the expect size of the input image?
- How many layers are in this network?
- How many hidden units are in this network?
- How many output units are in this network?

5. Which of the following are classification problems? Binary classification problems?
    - Finding traffic lights in an image
    - Determining the sex of a duck based on its height, weight, and other features
    - Determining the sex of a duck based on its photograph
    - Determining the maker of a car based on a photograph (out of the major car makers)
    - Translating from English to French

6. Consider this training loop. Which of the following are true?

```
for i in range(50):

    for (input, actual) in data:
        out = network(input)
        loss = criterion(out, actual)
        loss.backward()
        optim.step()
        optim.zero_grad()
```

    - The training data contains 50 items
    - The training data contains 1 item
    - Each training example is used once
    - Each training example is used 50 times
    - The optimizer used is gradient descent
    - The network is a 3-layer neural network
    - The variable name of the neural network is pigeon
    - The variable name of the neural network is network
    - The variable name of the neural network is input

Part 2: Model Training

"Training": (whether you "know" the material)

1. Sketch the ReLU, sigmoid, and tanh activation functions.
2. When training a neural network on a binary classification problem. Is it possible for the network's training accuracy to stay the same even while the loss function decreases?
3. Why do we not count the input layer when we count the number of layers in a neural network?
4. When training a neural network, what does the optimizer optimize? Does the optimizer optimize the *same* quantity in each iteration, or a different one? Explain.
5. What can happen when a batch size is too small? Too large?
6. How is a training curve helpful in detecting overfitting?
7. What are hyperparameters? What are some examples of hyperparameters?
8. How do we tune hyperparamters?
9. Why is checkpointing important when you train a neural network?
10. Label the purpose of each line in the training code below:

```
for epoch in range(num_epochs):
    for data, actual in iter(train_loader):
        out = model(data)
        loss = criterion(out, actual)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

Generalization: (whether you can apply the material)

1. Bob proposes to use a function f(x) = min(x, 0). Will this activation function make the network easier to train or harder to train? (Hint: the ReLU activation can also be written as relu(x) = max(x, 0))
2. Suppose you have a dataset of 10000 labeled training data. If you choose a batch size of 50, how many iterations are in an epoch? What about if you choose a batch size of 100?
3. You find that with a batch size of 64 and a learning rate of 0.0001, your neural network trains too slowly. What can you do?
4. One "hyperparameter" we haven't discussed is the initial (random) value of the parameters. Since neural network training does not find the *globally* optimal value of the parameters, the initial value of the parameters can be important. How can we tune this (set of) hyperparameters?
5. John and Sally are both training neural networks to classify cats vs dogs. John tuned his hyperparameters by training 20 different neural network models and choosing the best one. Sally tuned her

hyperparameters by training 2000 different neural network models and choosing the best one. For the following questions, assume that the validation and test sets are fairly large.

- Whose model do you think will have the better validation accuracy?
- Would you expect Sally's test accuracy be lower/higher than his validation accuracy?
- Would you expect John's test accuracy be lower/higher than his validation accuracy?
- Consider the difference between the test accuracy and the validation accuracy. Would you expect this difference to be higher for Sally or for John?

Part 3: Artificial Neural Networks

"Training": (whether you "know" the material)

1. What is the purpose of the softmax activation? How is it similar to the sigmoid activation? How is it different?
2. What is one technique to debug a neural network, to make sure that the programming is likely to be correct?

Generalization: (whether you can apply the material)

1. Identify 3 issues with the neural network model below:

```
class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        self.layer1 = nn.Linear(28 * 28, 40)
        self.layer2 = nn.Linear(30, 1)

    def forward(self, img):
        flattened = img.view(-1, 28 * 28)
        activation1 = self.layer1(flattened)
        activation2 = self.layer2(activation1)
        return torch.sigmoid(activation2)
```

2. Normally, the sigmoid or softmax activation in the last layer of the neural network is **not** applied in the `forward()` method. Why is that? (You will need to do some research for this question. You're not expected to understand the mathematics behind the reasoning, only the reasoning itself)

Part 4: Convolutional Neural Networks

"Training": (whether you "know" the material)

1. What are the advantages of convolutional layers over fully-connected layers?
2. PyTorch convolutions expect data in the "NCHW" format. What does this mean?
3. Why does it make sense to share weights in a convolutional neural network?
4. What is the purpose of zero padding in a convolutional layer?
5. Why should we design our neural networks so that the number of activations in each layer generally decreases?
6. Explain the idea of transfer learning, specifically of using AlexNet features to train a neural network more quickly.
7. What is the output of the following code?

```
>>> x = torch.randn(20, 3, 16, 16) # NCHW
>>> conv = nn.Conv2d(in_channels=3, out_channels=7, kernel_size=5,
padding=0)
>>> conv(x).shape
```

8. What new idea was introduced in GoogLeNet? ResNet?
9. What are fully-connected networks? Name one advantage and one disadvantage to using a fully convolutional network.

Generalization: (whether you can apply the material)

1. How many multiplications are performed when applying a 3x3 convolution on a greyscale image of size 7x7?
2. Which operations to reduce the number of units in the hidden layer are the most and least computationally efficient: max pooling, average pooling, and using strides in a convolution?
3. Suppose we learn a fully-connected network on a 128x128 image. Can we use the same network to make predictions about images of size 256x256? Why or why not?

Part 5: Autoencoders

"Training": (whether you "know" the material)

1. Describe the idea of a "distributed" encoding.
2. What is data augmentation?
3. Describe the following strategies for preventing overfitting: data augmentation, data normalization, model averaging, dropout, weight decay, and early stopping.
4. In PyTorch, why is it important to mark whether a model is currently used for training or for testing?
5. Describe, intuitively, why large weights are indicative of overfitting.
6. What is an autoencoder?
7. What is the loss function used when training an autoencoder?
8. What is an "embedding"?

Generalization: (whether you can apply the material)

1. Consider augmenting an image dataset by shifting the training images by a random (small) number of pixels. For which type of neural network would this type of data augmentation have a bigger impact: a convolutional network or a fully-connected network?
2. Suppose you have two networks: network A has 10 million hidden units (artificial neurons) and 1 million weights, network B has 10 million weights and 1 million hidden units. Which of the two networks has higher capacity? Which of the two networks is more likely to overfit?
3. In PyTorch, the implementation of weight decay in an optimizer (e.g. `optim.SGD(model.parameters(), weight_decay=0.0001)`) performs weight decay for all parameters, including biases. Why might we want to allow large biases, and only decay neural network weights?
4. We discussed data augmentation for images. Suppose that you are instead working with voice recordings. What are some ways to augment a dataset of voice recordings?
5. What do you think would happen if the encoder output of an autoencoder is larger than the image size? You may assume that both the encoder and decoder have high capacity.
6. What do you think would happen if the encoder output of an autoencoder is too small?
7. What does overfitting look like in the context of an autoencoder?
8. Why do we use the sigmoid activation in an autoencoder of images?
9. Suppose that an autoencoder used only convolutional layers, with no fully connected layers, and also no global pooling layers. Can the autoencoder be used to generate images of different sizes? If so how?
10. How can you use autoencoders for data augmentations?